

Object Oriented Programming (CS1004)

Course Instructor(s):

Mr. Pir Sami Ullah Shah, Ms. Mahnoor Ayaz

Section(s): SE (A, B, C, D)

Sessional-II Exam

Total Time (Hrs): 1

Total Marks: 55

Total Questions: 1

Date: Apr 15, 2026

Roll No

Course Section

Student Signature

Do not write below this line.

Attempt all the questions.

[CLO1: Demonstrate the basic concepts of OOP]

Q1: What would be the output produced by executing the following C++ codes? Identify errors, if any (identify, and fix error, and write the output otherwise marks will not be awarded). All the code snippet contains #include and using namespace std while these programs are compiled using g++ on a 32-bit system architecture. **[5 + 5 + 5 + 5 + 5 + 5 + 5 + 5 + 10 + 10 = 55 marks]**

1.		Output [5 Marks]
	<pre> class Tricky { int a, b; public: Tricky(int x = 2, int y = 3) : b(x + a), a(x * y) { cout << "a=" << a << " b=" << b << endl; } Tricky(const Tricky& o) : b(o.a + 1), a(o.b + 1) { cout << "Copy: a=" << a << " b=" << b << endl; } void show() const { cout << a << " " << b << endl; } }; int main() { Tricky t1; Tricky t2(t1); Tricky t3 = t2; Tricky t4(3, 2); Tricky t5(t4); t5.show(); } </pre>	<pre> a=6 b=8 Copy: a=9 b=7 Copy: a=8 b=10 a=6 b=9 Copy: a=10 b=7 10 7 </pre>

National University of Computer and Emerging Sciences

Islamabad Campus

2.	<pre> class DeepVsShallow { int* data; int size; public: DeepVsShallow(int s) : size(s) { data = new int[size]; for (int i = 0; i < size; i++) data[i] = i * 10; cout << "Constructed, size=" << size << endl; } DeepVsShallow(const DeepVsShallow& other) : size(other.size) { data = other.data; cout << "Copied, size=" << size << endl; } void modify(int idx, int val) { data[idx] = val; } void display() const { for (int i = 0; i < size; i++) cout << data[i] << " "; cout << endl; } ~DeepVsShallow() { cout << "Destroyed, data[0]=" << data[0] << endl; delete[] data; } }; void process(DeepVsShallow obj) { obj.modify(0, 999); obj.display(); } int main() { DeepVsShallow d(3); process(d); d.display(); } </pre>	Output [5 Marks] Constructed, size=3 Copied, size=3 999 10 20 Destroyed, data[0]=999 23869 0 1971692770 Destroyed, data[0]=23869 free(): double free detected in tcache 2 Aborted
3.	<pre> class Car { static int count; int id; int* speed; public: Car(int s) { count++; id = count; speed = new int(s); cout << "Car#" << id << " speed=" << *speed << endl; } Car(const Car& c) { count++; id = count; speed = new int(*c.speed + 10); } } </pre>	Output [5 Marks] Car#1 speed=50 Car#2 speed=60 Assigned#2 speed=55 Car#3 speed=80 Total: 3 80 ~Car#3 speed=80 ~Car#2 speed=55 ~Car#1 speed=50

National University of Computer and Emerging Sciences

Islamabad Campus

	<pre> cout << "CarCopy#" << id << " speed=" << *speed << endl; } Car& operator=(const Car& c) { if (this != &c) { *speed = *c.speed + 5; cout << "Assigned#" << id << " speed=" << *speed << endl; } return *this; } ~Car() { cout << "~Car#" << id << " speed=" << *speed << endl; delete speed; } static int getCount() { return count; } int getSpeed() const { return *speed; } }; int Car::count = 0; Car makecar() { Car temp(80); return temp; } int main() { Car c1(50), c2(60); c2 = c1; Car c3 = makecar(); cout << "Total: " << Car::getCount() << endl; cout << c3.getSpeed() << endl; } </pre>	
4.	<pre> class Token { static int next; int id; char type; public: Token() : id(++next), type('D') { cout << "Token " << id << " Default" << endl; } Token(char t) : id(++next), type(t) { cout << "Token " << id << " type=" << type << endl; } Token(const Token& o) : id(++next), type(o.type + 1) { cout << "Token " << id << " CopiedFrom=" << o.id << " type=" << type << endl; } ~Token() { cout << "~Token " << id << " type=" << type << endl; } }; </pre>	Output [5 Marks] Token 1 Default Token 2 Default Token 3 Default Token 4 type=A Token 5 CopiedFrom=4 type=B ---end--- ~Token 5 type=B ~Token 4 type=A

National University of Computer and Emerging Sciences
Islamabad Campus

	<pre>int Token::next = 0; int main() { Token arr[3]; Token x('A'); Token y = x; arr[1] = y; cout << "---end---" << endl; }</pre>	~Token 3 type=D ~Token 5 type=B ~Token 1 type=D
5.	<pre>class Grid { int data[3][3]; public: Grid() { for(int i=0;i<3;i++) for(int j=0;j<3;j++) data[i][j] = i*3+j; cout << "Grid Ctor" << endl; } int* operator[](int row) { cout << "[] row=" << row << endl; return data[row]; } int operator()(int r, int c) const { cout << "() r=" << r << " c=" << c << endl; return data[r][c]; } Grid& operator=(const Grid& o) { cout << "Grid =" << endl; for(int i=0;i<3;i++) for(int j=0;j<3;j++) data[i][j] = o.data[i][j]; return *this; } friend ostream& operator<<(ostream& os, const Grid& g) { for(int i=0;i<3;i++){ for(int j=0;j<3;j++) os << g.data[i][j] << " "; os << endl; } return os; } }; int main() { Grid g1, g2; g1[1][1] = 99; cout << g1(1,1) << endl;</pre>	Output [5 Marks] Grid Ctor Grid Ctor [] row=1 () r=1 c=1 99 Grid = 0 1 2 3 99 5 6 7 8

National University of Computer and Emerging Sciences
Islamabad Campus

	<pre> g2 = g1; cout << g2; } </pre>	
6.	<pre> class Base { public: int val; Base(int v = 0) : val(v) { cout << "Base(" << val << ")" << endl; } virtual ~Base() { cout << "~Base(" << val << ")" << endl; } virtual void show() { cout << "Base::show val=" << val << endl; } }; class Left : virtual public Base { public: Left(int v) : Base(v) { cout << "Left(" << v << ")" << endl; } ~Left() { cout << "~Left" << endl; } virtual void show() { cout << "Left::show" << endl; } }; class Right : virtual public Base { public: Right(int v) : Base(v) { cout << "Right(" << v << ")" << endl; } ~Right() { cout << "~Right" << endl; } virtual void show() { cout << "Right::show" << endl; } }; class Diamond : public Left, public Right { public: Diamond(int v) : Base(v), Left(v+1), Right(v+2) { cout << "Diamond(" << v << ")" << endl; } ~Diamond() { cout << "~Diamond" << endl; } void show() { cout << "Diamond::show val=" << val << endl; } }; int main() { Diamond d(5); d.show(); Left* lp = &d; lp->show(); Base* bp = &d; bp->show(); cout << "val=" << bp->val << endl; } </pre>	Output [5 Marks] Base(5) Left(6) Right(7) Diamond(5) Diamond::show val=5 Diamond::show val=5 Diamond::show val=5 val=5 ~Diamond ~Right ~Left ~Base(5)

National University of Computer and Emerging Sciences

Islamabad Campus

7.	<pre> class Base { public: int x; Base(int x_ = 0) : x(x_) {} virtual void vfunc() { cout << "Base::vfunc x=" << x << endl; } void nfunc() { cout << "Base::nfunc x=" << x << endl; } ~Base() { cout << "~Base" << endl; } }; class Derived : public Base { public: int y; Derived(int x_, int y_) : Base(x_), y(y_) {} void vfunc() { cout << "Derived::vfunc x=" << x << " y=" << y << endl; } void nfunc() { cout << "Derived::nfunc y=" << y << endl; } ~Derived() { cout << "~Derived" << endl; } }; void byValue(Base b) { b.vfunc(); b.nfunc(); } void byPointer(Base* b) { b->vfunc(); b->nfunc(); } void byRef(Base& b) { b.vfunc(); b.nfunc(); } int main() { Derived d(10, 20); cout << "--- by value ---" << endl; byValue(d); cout << "--- by pointer ---" << endl; byPointer(&d); cout << "--- by ref ---" << endl; byRef(d); } </pre>	Output [5 Marks] <pre> --- by value --- Base::vfunc x=10 Base::nfunc x=10 ~Base --- by pointer --- Derived::vfunc x=10 y=20 Base::nfunc x=10 --- by ref --- Derived::vfunc x=10 y=20 Base::nfunc x=10 ~Derived ~Base </pre>
----	---	---

National University of Computer and Emerging Sciences

Islamabad Campus

8.	<pre> class Shape { protected: string name; public: Shape(string n) : name(n) { cout << "Shape: " << name << endl; } virtual double area() const = 0; virtual void describe() const { cout << name << " area=" << area() << endl; } virtual ~Shape() { cout << "~Shape: " << name << endl; } }; class Circle : public Shape { double r; public: Circle(double r_) : Shape("Circle"), r(r_) { cout << "Circle r=" << r << endl; } double area() const { return 3.14 * r * r; } ~Circle() { cout << "~Circle" << endl; } }; class Rect : public Shape { double w, h; public: Rect(double w_, double h_) : Shape("Rect"), w(w_), h(h_) { cout << "Rect " << w << "x" << h << endl; } double area() const { return w * h; } void describe() const { cout << "Rectangle: " << w << "x" << h << " area=" << area() << endl; } ~Rect() { cout << "~Rect" << endl; } }; int main() { Shape* shapes[3]; shapes[0] = new Circle(7); shapes[1] = new Rect(4, 5); shapes[2] = new Circle(3); for (int i = 0; i < 3; i++) { shapes[i]->describe(); } cout << "--- cleanup ---" << endl; for (int i = 0; i < 3; i++) delete shapes[i]; } </pre>	Output[10 Marks] <pre> Shape: Circle Circle r=7 Shape: Rect Rect 4x5 Shape: Circle Circle r=3 Circle area=153.86 Rectangle: 4x5 area=20 Circle area=28.26 --- cleanup --- ~Circle ~Shape: Circle ~Rect ~Shape: Rect ~Circle ~Shape: Circle </pre>
----	--	---

National University of Computer and Emerging Sciences
Islamabad Campus

9.		Output [10 Marks]
	<pre> class Entity { protected: int id; static int counter; public: Entity() : id(++counter) { cout << "Entity#" << id << endl; } Entity(const Entity& o) : id(++counter) { cout << "Entity#" << id << " copied from #" << o.id << endl; } virtual void identify() const { cout << "Entity#" << id << endl; } virtual Entity* clone() const { return new Entity(*this); } virtual ~Entity() { cout << "~Entity#" << id << endl; } }; int Entity::counter = 0; class Agent : virtual public Entity { protected: string role; public: Agent(string r = "none") : role(r) { cout << "Agent role=" << role << endl; } Agent(const Agent& o) : Entity(o), role(o.role + "_copy") { cout << "Agent copied role=" << role << endl; } void identify() const { cout << "Agent#" << id << " role=" << role << endl; } Agent* clone() const { return new Agent(*this); } ~Agent() { cout << "~Agent role=" << role << endl; } }; class Supervisor : virtual public Entity { protected: int level; public: Supervisor(int l = 1) : level(l) { cout << "Supervisor level=" << level << endl; } void identify() const { cout << "Supervisor#" << id << " level=" << level << endl; } ~Supervisor() { cout << "~Supervisor level=" << level << endl; } }; class Manager : public Agent, public Supervisor { public: Manager(string r, int l) : Entity(), Agent(r), Supervisor(l) { cout << "Manager" << endl; } void identify() const { </pre>	<pre> Entity#1 Agent role=CEO Supervisor level=5 Manager Manager#1 role=CEO level=5 Manager#1 role=CEO level=5 Entity#2 Agent role=Dev Entity#3 copied from #2 Agent copied role=Dev_copy Agent#3 role=Dev_copy --- cleanup --- ~Agent role=Dev ~Entity#2 ~Agent role=Dev_copy ~Entity#3 ~Manager ~Supervisor level=5 ~Agent role=CEO ~Entity#1 </pre>

National University of Computer and Emerging Sciences
Islamabad Campus

<pre> cout << "Manager#" << id << " role=" << role << " level=" << level << endl; } ~Manager() { cout << "~Manager" << endl; } }; int main() { Manager m("CEO", 5); m.identify(); Entity* e = &m; e->identify(); Agent* a = new Agent("Dev"); Entity* ec = a->clone(); ec->identify(); cout << "--- cleanup ---" << endl; delete a; delete ec; }</pre>	
--	--